

Principaux types de données

Données numériques

- Type *integer*

```
7  
8 for i in range(50):  
9     print(i, ":", type(i))
```

- Type *float*

Fonction range()

- utilisée pour générer une séquence d'entiers
- fréquemment utilisée dans les boucles *for* pour itérer sur des valeurs spécifiques.
- offre une flexibilité pour contrôler le début, la fin et l'incrément de la séquence générée.
- retourne une liste de nombres en prenant de 1 à 3 entiers :
 - start : un entier qui détermine le premier élément dans la séquence de nombre
 - stop : l'entier jusqu'auquel on va boucler mais sans l'inclure (comme on commence à compter à partir de 0)
 - step : un entier qui peut être positif ou négatif pour définir de combien on va incrémenter entre chaque élément de la liste

Examples

range(3)

```
for i in range(3):  
    print(i)
```

0
1
2

=> [0, 1, 2]

range(2,8)

```
for i in range(2, 8):  
    print(i)
```

2
3
4
5
6
7

=> [2, 3, 4, 5, 6, 7]

range(3, 10, 2)

```
for i in range(0, 42, 7):  
    print(i)
```

0
7
14
21
28
35

=> [3, 5, 7, 9]

Exemples

```
for i in range(10, 0, -1):  
    print(i)
```

10 est le nombre avec lequel on commence, inclus

0 est notre nombre de fin, non inclus

-1 est le saut qu'on va faire en partant de notre nombre de début jusqu'à notre nombre de fin (non-inclus)

```
for i in range(42, 0, -7):  
    print(i)
```

42

35

28

21

14

7

String

```
>>> phrase1 = 'les oeufs durs.'
>>> phrase2 = '"Oui", répondit-il,'
>>> phrase3 = "j'aime bien"
>>> print(phrase2, phrase3, phrase1)
"Oui", répondit-il, j'aime bien les oeufs durs.
```

*Le caractère spécial *

- Permet d'écrire sur plusieurs lignes une commande qui serait trop longue pour tenir sur une seule
- À l'intérieur d'une chaîne de caractères, il permet d'insérer un certain nombre de codes spéciaux (apostrophes, sauts de ligne, guillemets...)

```
>>> txt3 = '"N\'est-ce pas ?" répondit-elle.'
>>> print(txt3)
"N'est-ce pas ?" répondit-elle.
>>> Salut = "Ceci est une chaîne plutôt longue\n contenant plusieurs lignes \
... de texte (Ceci fonctionne\n de la même façon en C/C++.\n\
...     Notez que les blancs en début\n de ligne sont significatifs.\n"
>>> print(Salut)
Ceci est une chaîne plutôt longue
contenant plusieurs lignes de texte (Ceci fonctionne
de la même façon en C/C++.
    Notez que les blancs en début
de ligne sont significatifs.
```

Triple quotes

- Pour insérer plus aisément des caractères spéciaux ou exotiques dans une chaîne, sans utiliser `\` ou pour faire accepter `\` lui-même dans la chaîne :

```
>>> a1 = """
... Usage: trucmuche[OPTIONS]
... { -h
...   -H hôte
... }"""
```

```
>>> print(a1)
```

```
Usage: trucmuche[OPTIONS]
{ -h
  -H hôte
}
```

Accès aux caractères individuels d'une chaîne

- Chaînes de car. font partie des données appelées **données composites** :
 - une entité qui rassemble dans une seule structure un **ensemble d'entités plus simples**
 - dans le cas d'une chaîne de car. il s'agit des **caractères** eux-mêmes
- Caractères d'une chaîne sont disposées toujours dans **un certain ordre**

```
>>> ch = "Christine"  
>>> print(ch[0], ch[3], ch[5])
```

- Indiçage, extraction, longueur

```
>>> nom = 'Cédric'
>>> print(nom[1], nom[3], nom[5])
é r c
>>> print(nom[-1], nom[-2], nom[-4], nom[-6])
c i d C
>>>
>>> print(len(nom))
6
```

- Extraction de fragments de chaînes

```
>>> print(ch[:3])           # les 3 premiers caractères
Jul
>>> print(ch[3:])          # tout ce qui suit les 3 premiers caractères
iette
```

Opérations sur les chaînes

- Concaténation, répétition

```
a = 'Petit poisson'  
b = ' deviendra grand'  
c = a + b  
print(c)
```

```
>>> n = 'abc' + 'def'          # concaténation  
>>> m = 'zut !' * 4           # répétition  
>>> print(n, m)  
abcdef zut ! zut ! zut ! zut !
```

Appartenance d'un élément à une séquence : l'instruction *in* utilisée seule

```
car = "e"  
voyelles = "aeiouyAEIOUYàâéèêëùîï"  
if car in voyelles:  
    print(car, "est une voyelle")
```

Les chaînes sont des séquences non modifiables

- Tester :

```
salut = 'bonjour à tous'  
salut[0] = 'B'  
print(salut)
```

```
salut = 'bonjour à tous'  
salut = 'B' + salut[1:]  
print(salut)
```

Dans cet autre exemple, nous ne modifions pas la chaîne **salut**, mais en créons une nouvelle, avec le même nom.

Les chaînes sont comparables

- utile pour trier par ordre alphabétique

```
7
8 n="hello"
9 if n<"salut":
10     place="précède"
11 elif n>"salut":
12     place="suit"
13 else:
14     place="se confond avec"
15 print("le mot", n, place, "le mot 'salut' dans l'ordre alphabétique")
```

- ne **fonctionne** bien que pour des mots qui sont **tous entièrement en minuscules, ou entièrement en majuscules**, et qui **ne comportent aucun caractère accentué** car les majuscules et minuscules utilisent des ensembles de codes distincts.
- les caractères accentués sont encodés en dehors de l'ensemble constitué par les caractères du standard *ASCII*.

Fonctions prédéfinies

print()

- Séparateur par défaut est l'espace

```
a=3
b=2
print(a,b)
```

- On peut le remplacer avec l'argument « sep »

```
>>> print("Bonjour", "à", "tous", sep="*")
>>> print("Bonjour", "à", "tous", sep="")
```

- On peut remplacer le saut à la ligne terminal avec l'argument « end »

```
>>> n = 0
>>> while n < 6:
...     print("zut", end="")
...     n = n + 1
...
zutzutzutzutzut
```

input ()

- Interaction avec l'utilisateur
- Cette fonction provoque une interruption dans le programme courant et l'utilisateur est invité à entrer des caractères au clavier et à terminer avec <Enter>

```
prenom = input("Entrez votre prénom : ")  
print("Bonjour,", prenom)
```

- La fonction renvoie toujours une chaîne de car.

```
>>> a = input("Entrez une donnée numérique : ")  
Entrez une donnée numérique : 52.37  
>>> type(a)  
<class 'str'>  
>>> b = float(a)           # conversion de la chaîne en un nombre réel  
>>> type(b)  
<class 'float'>
```

Fonctions intégrées

- **len(ch)** renvoie la longueur de la chaine (nombre de caractères)

```
>>> ch = 'Georges'
>>> print(len(ch))
7
```

- **float(ch)** convertit la chaine en un nombre réel (*float*)

```
>>> a = float("12.36") # Attention : pas de virgule décimale !
>>> print(a + 5)
17.36
```

- **int(ch)** convertit la chaine **ch** en un nombre entier

```
>>> a = int("184")
>>> print(a + 20)
204
```

- **str(obj)** convertit (ou représente) l'objet **obj** en une chaine de caractères. **obj** peut être une donnée d'à peu près n'importe quel type :

```
>>>> a, b = 17, ["Emile", 7.65]
>>> ch = str(a) + " est un entier et " + str(b) + " est une liste."
>>> print(ch)
17 est un entier et ['Emile', 7.65] est une liste.
```

```
>>> ch = '8647'
>>> print(ch + 45)
```

→

```
>>> n = int(ch)
>>> print(n + 65)
```

8712

Les chaînes sont des objets :
méthodes appropriées

split() : segmenteur

- convertit **une chaine en une liste de sous-chaines**.
- on peut choisir le **caractère séparateur** en le fournissant comme argument, sinon c'est un espace par défaut :

```
>>> c2 = "Votez pour moi"
```

```
>>> a = c2.split()
```

```
>>> print(a)
```

```
['Votez', 'pour', 'moi']
```

```
>>> c4 = "Cet exemple, parmi d'autres, peut encore servir"
```

```
>>> c4.split(",")
```

```
['Cet exemple', " parmi d'autres", ' peut encore servir']
```

join(liste) :

- **rassemble une liste de chaines en une seule.**
- la chaine à laquelle on applique cette méthode est celle qui servira de séparateur;
- l'argument transmis est la liste des chaines à rassembler :

```
>>> b=["Il", "a", "mangé", "du", "foin"]
```

```
> >> print(" ".join(b))
```

Il a mangé du foin

```
>>> print("---".join(b))
```

Il---a---manger---du---foin

- **find(sch)** : cherche la position d'une sous-chaîne dans la chaîne :

```
>>> ch1 = "Cette leçon vaut bien un fromage, sans doute ?"
```

```
>>> ch2 = "fromage"
```

```
>>> print(ch1.find(ch2))
```

```
25
```

- **count(sch)** : compte le nombre de sous-chaînes dans la chaîne

```
>>> ch1 = "Le héron au long bec emmanche d'un long cou"
```

```
>>> ch2 = 'long'
```

```
>>> print(ch1.count(ch2))
```

```
2
```

- **lower()** : convertit une chaîne en minuscules :
 >>> ch = "CELMENE est un prenom ancien"
 >>> print(ch.lower())
 celimene est un prenom ancien
- **upper()** : convertit une chaîne en majuscules
- **title()** : convertit en majuscule l'initiale de chaque mot :
 >>> ch = "albert rene elise veronique"
 >>> print(ch.title())
 Albert Rene Elise Veronique
- **capitalize()** : convertit en majuscule seulement la première lettre de la chaîne :
 >>> b3 = "quel beau temps, aujourd'hui !"
 >>> print(b3.capitalize())
 "Quel beau temps, aujourd'hui !"
- **swapcase()** : convertit toutes les majuscules en minuscules, et vice-versa :
 >>> ch = "Le Lievre Et La Tortue"
 >>> print(ch.swapcase())
 lE lIEVRE eT lA tORTUE

- **strip()** : enlève les espaces éventuels au début et à la fin de la chaîne :

```
>>> ch = " Monty Python "
```

```
>>> ch.strip()
```

```
'Monty Python'
```

- **replace(c1, c2)** : remplace tous les caractères **c1** par des caractères **c2** dans la chaîne :

```
>>> ch8 = "Si ce n'est toi c'est donc ton frère"
```

```
>>> print(ch8.replace(" ", "*"))
```

```
Si*ce*n'est*toi*c'est*donc*ton*frère
```

- **index(car)** : retrouve l'indice (*index*) de la première occurrence du caractère dans la chaîne :

```
>>> ch9 = "Portez ce vieux whisky au juge blond qui fume"
```

```
>>> print(ch9.index("w"))
```

```
16
```


Méthode index vs fonction find

```
>>> ch9 = "Portez ce vieux whisky au juge blond qui fume"
```

```
>>> print(ch9.index("e"))      # cherche à partir du début de la chaîne  
4                             # et trouve le premier 'e'  
>>> print(ch9.index("e",5))   # cherche seulement à partir de l'indice 5  
8                             # et trouve le second 'e'  
>>> print(ch9.index("e",15))  # cherche à partir du caractère n° 15  
29                             # et trouve le quatrième 'e'
```

- Comparez avec une fonction find()

```
>>> ch1 = "Cette leçon vaut bien un fromage, sans doute ?"  
>>> ch2 = "fromage"  
>>> print(ch1.find(ch2))  
25
```

Formatage des chaînes de caractères

- **construire une chaîne de caractères complexe à partir d'un certain nombre de morceaux**, tels que les valeurs de variables diverses.
- programme qui traite de la couleur et de la température d'une solution aqueuse, en chimie.
 - La couleur est mémorisée dans une chaîne de caractères nommée **coul**,
 - la température est mémorisée dans une variable de type réel nommée **temp**.
 - on veut que le programme construise une chaîne de caractères à partir de ces données

Exemple : une phrase « La solution est devenue **rouge**, et sa température atteint **12,7 °C** ».

En bref

coul='rouge'

temp='12.7'

=> « La solution est devenue **rouge**, et sa température atteint **12.7 °C** »

Formatage des chaînes de caractères

1ère solution :

- assembler des morceaux à l'aide de l'opérateur de concaténation

Attention : ne pas oublier de convertir en chaîne de caractères la valeur numérique de type *float* à l'aide de la fonction intégrée **str()**

```
coul="verte"  
temp=12.7  
print("La solution est devenue", coul, "et sa température atteint", str(temp), "°C ")
```

ou

```
coul="verte"  
temp=12.7  
print("La solution est devenue " + coul + " et sa température atteint " + str(temp) + "°C ")
```

Formatage des chaînes de caractères

2^{ème} solution :

- **préparer une chaîne « patron »** contenant l'essentiel du texte invariable, avec des *balises* particulières aux endroits (*les champs*) où vous souhaitez qu'apparaissent des contenus variables.
- **appliquer** ensuite à cette chaîne la **méthode format()**, à laquelle on fournit comme arguments les divers objets à convertir en caractères et à insérer en remplacement des balises.

```
>>> coul = "verte"
>>> temp = 1.347 + 15.9
>>> ch = "La couleur est {} et la température vaut {} °C"
>>> print(ch.format(coul, temp))
La couleur est verte et la température vaut 17.247 °C
```

Formatage des chaînes de caractères

3^{ème} solution : Formatage des chaînes « à l'ancienne »

- assembler deux éléments à l'aide de l'opérateur % .
- A gauche de cet opérateur, la chaîne « patron » contenant des balises commençant toujours par %,
- à droite (entre parenthèses) le ou les objets que Python devra insérer dans la chaîne, en lieu et place des balises.

```
>>> coul = "verte"
>>> temp = 1.347 + 15.9
>>> print("La couleur est %s et la température vaut %s °C" % (coul, temp))
La couleur est verte et la température vaut 17.247 °C
```

Véracité/fausseté d'une expression

- Un ordinateur ne manipule que des nombres => **Tout ce qu'un ordinateur doit traiter doit être converti en valeur numérique**
- Toute valeur numérique autre que 0 est « vraie ». Seule la valeur 0 est « fausse »

Véracité/fausseté d'une expression

- Testons le caractère « vrai » ou « faux » d'un nombre entier

```
ch = input('Entrez un nombre entier quelconque')
n = int(ch)
if n:
    print("vrai")
else:
    print("faux")
```

Ce script n'affiche « faux » que si vous entrez la valeur 0.

Astuce :

if n: désigne $n \neq 0$

Véracité/fausseté d'une expression

- Testons le caractère « vrai » ou « faux » d'une chaîne de car.

```
ch = input("Entrez une chaîne de caractères quelconque")
if ch:
    print("vrai")
else:
    print("faux")
```

Ce script affiche « faux » pour toute chaîne vide et « vrai » pour toute chaîne contenant au moins un caractère.

L'instruction **if ch:** est équivalente à **if ch!=" ":** pour nous (humains)

Véracité/fausseté d'une expression

```
ch =input("Veuillez entrer un nombre : ")
n =int(ch)
if n % 2:                #si c'est vrai, donc !=0, le nombre entré est impair
    print("Il s'agit d'un nombre impair.")
else:
    print("Il s'agit d'un nombre pair.")
```

Boucle 'while True:'

```
while True:
    mot = input("Entrez un mot quelconque : (<enter> pour terminer)")
    if mot == "":
        break
    if mot < "limonade":
        place = "précède"
    elif mot > "limonade":
        place = "suit"
    else:
        place = "se confond avec"
    print("Le mot", mot, place, "le mot 'limonade' dans l'ordre alphabétique")
```

- Une chaîne vide est égale à faux. Si on ne tape rien (enter), la boucle s'arrête.
- La boucle 'while True:' s'arrête avec, à l'intérieur, une condition qui déclenche un 'break': c'est le break qui permet de sortir de la boucle
- while machin==truc:
machin==truc est évalué; et en fonction du résultat, la boucle while continue ou s'arrête.
python évalue une condition à True si elle est vrai ou à False.

Mots-clefs *break*

- Break : permet d'interrompre une boucle

while 1: #1 est toujours vrai->boucle infini

```
    lettre=input("tapiez une lettre ou 'Q' pour  
quitter :")
```

```
    if lettre=="Q":
```

```
        print("fin de la boucle")
```

```
        break
```

- La boucle *while* a pour condition 1 : une condition sera toujours vraie.
 - On demande à l'utilisateur de taper une lettre.
 - Si c'est Q, le programme affiche « fin de la boucle » et la boucle s'arrête grâce au mot clé *break*,
 - sinon le programme va redemander de taper une lettre

Mot clé *continue*

```
i=1
```

```
while i<20:
```

```
    if i%3==0:
```

```
        i+=4        #on ajoute 4 à i
```

```
        print("on incrémente i de 4. i est maintenant égale à", i)
```

```
        continue #on retourne au while sans exécuter les autres lignes
```

```
    print ("la variable i=", i)
```

```
    i+=1
```

Arrivé au mot clé *continue* Python n'exécute pas la fin du bloc mais revient au début de la boucle en testant à nouveau la condition du *while*.

On y reviendra

Travail à faire

- On a 7 prénoms : *Jack, Kack, Lack, Mack, Nack, Oack, Pack et Qack*. Ecrivez un petit script qui génère tous ces noms à partir des deux chaînes suivantes : **prefixes** = 'JKLMNOPQ' et **suffixe** = 'ack'
- Ecrire un script qui détermine si une chaîne contient ou non le caractère « e »
- Ecrire un script qui parcourt une chaîne : lorsqu'il rencontre une voyelle, il l'affiche, si c'est une consonne, il met « * »
- Ecrire un script qui recopie une chaîne (dans une nouvelle variable), en insérant des astérisques entre les caractères
 - Exemple : « iris » devra devenir « i*r*i*s »
- Ecrire un script qui recopie une chaîne (dans une nouvelle variable) en l'inversant
 - Exemple : « iris » devra devenir « siri »

Travail à faire (pour les avancés)

Jeu de Juste prix : Le but est de deviner un nombre entier choisi aléatoirement par Python.

- Écrivez un script qui attribue un nombre à une variable.
- Dans une boucle, demandez un input à l'utilisateur et informez le joueur si le nombre entré est égale, plus grand ou plus petit que le juste prix.

```
import random #génère des nombres d'une manière aléatoire
prix = random.randint(1, 10)
tentatives = 0
print("Devinez le juste prix ! Le prix est un nombre compris
entre 1 et 10 inclus.")
```